Classes and Objects

Module-1



Dr.M. Sivakumar

Professor & Technical Trainer / CSE

Saveetha School of Engineering, SIMATS

sivakumarm.sse@saveetha.com

+91-9500600868

Outline

- ✓ Class
- ✓ Object
- ✓ Methods
- ✓ Constructors
- ✓ Access Specifiers
- ✓ Static Members

Class

- Class is **derived datatype**, it combines members of different datatypes into one.
- Defines new datatype (primitive ones are not enough).
  - For Example : **Car, College, Bus etc..**
- This new datatype can be used to create objects.
- A class is a template for an object .

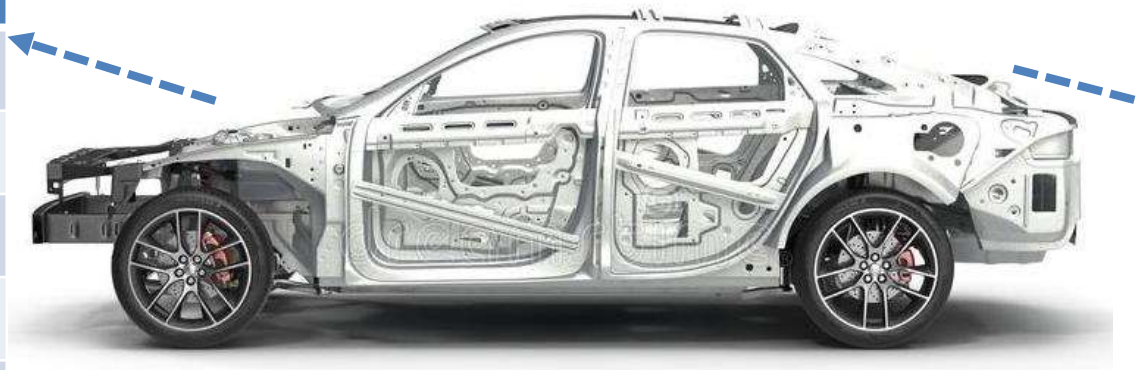
Example :

```
class Car{
        String company;
        String model;
        double price;
        double milage;
        ……
}
```

Car Class

Class: Car

Properties (Describe)
Company
Model
Color
Mfg. Year
Price
Fuel Type
Mileage
Gear Type
Power Steering
Anti-Lock braking system



Methods (Functions)
`Start`
`Drive`
`Park`
`On_break`
`On_lock`
`On_turn`

Object

- An object is an **instance** of a **class**.
- An object has a **state** and **behavior**.

  Example: A dog has

  states - color, name, breed as well as

  behaviors – wagging the tail, barking, eating.
- The **state** of an object is stored in **fields** (variables), while **methods** (functions) display the object's **behavior**.

Objects of Class Bird







Objects of a class Car



Honda City



Hyundai i20



Sumo Grand



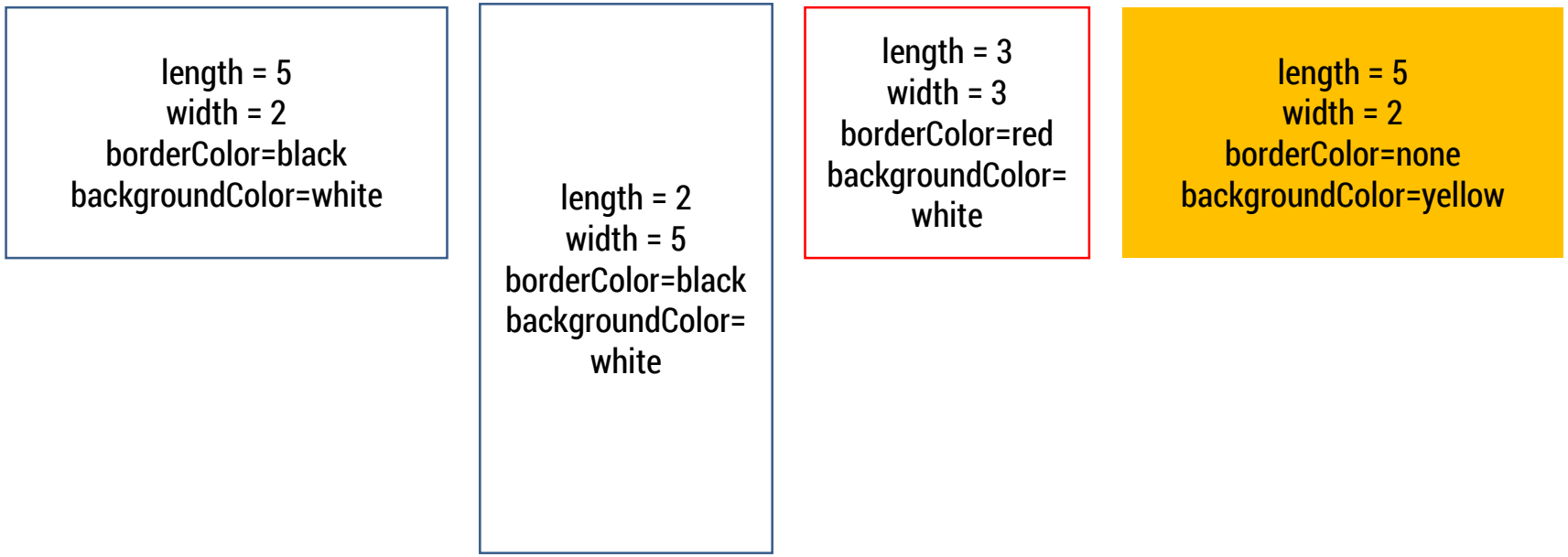
Mercedes E class



Swift Dzire

Rectangle Class

- Properties in rectangle Class
  - length
  - width
  - borderColor
  - backgroundColor



Creating Object & Accessing members

- **new** keyword creates new object
- Syntax:

  ClassName objName = **new** ClassName();

  Example :

  SmartPhone iPhone = **new** SmartPhone();
- Object variables and methods can be accessed using the **dot (.)** operator
- Example:

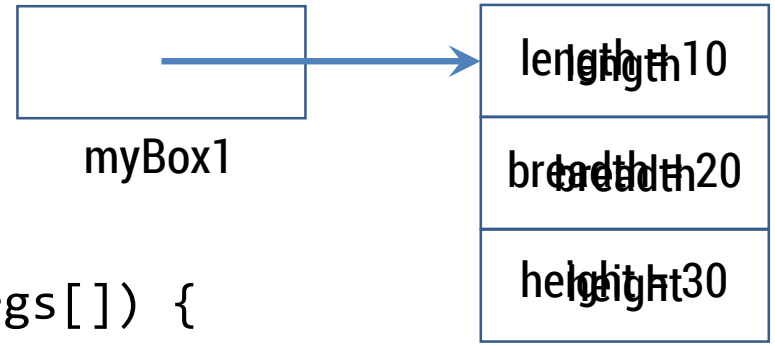
  iPhone.storage = 8000;

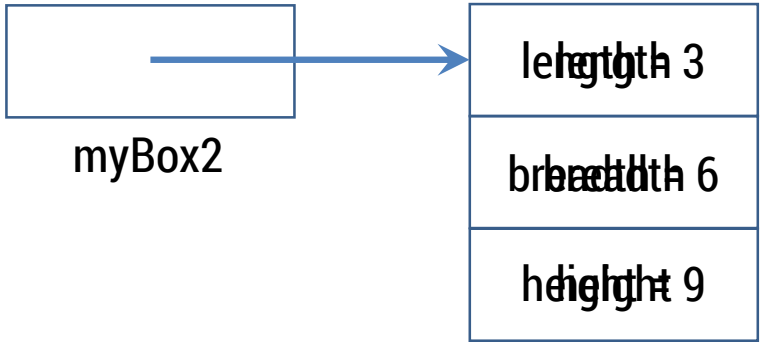
```
class Box {
        double length; // Members
        double breadth;
        double height;
}
class BoxDemo {
        public static void main(String args[]) {
                Box myBox1 = new Box();
                Box myBox2 = new Box();
                double vol;

                myBox1.length = 10;
                myBox1.breadth = 20;
                myBox1.height = 30;

                myBox2.length = 3;
                myBox2.breadth = 6;
                myBox2.height = 9;

                vol = myBox1.length * myBox1.breadth * myBox1.height;
                System.out.println("Volume is " + vol);
                vol = myBox2.length * myBox2.breadth * myBox2.height;
                System.out.println("Volume is " + vol);
        }
}
```





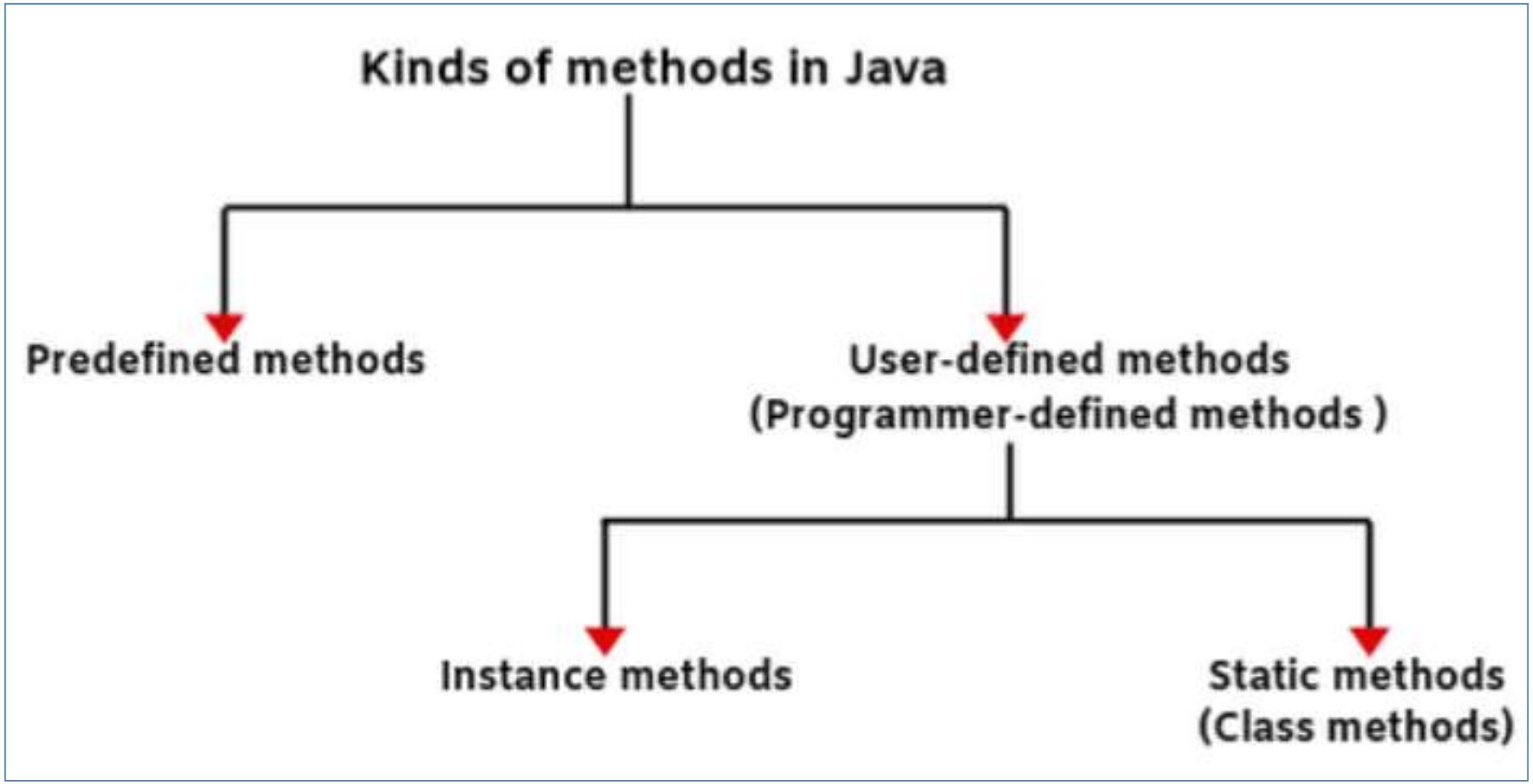
Methods

- Methods represents the **behavior** of a class.
- Remember : a Method is always invoked relative to an object. (except ….)
- Syntax:

```
access_specifier return_type method_name(argument_list)
{
    // code
}
```

- Methods must be defined inside the class.
- To access any method from a class, use object with .(dot) operator.

Methods


Kinds of methods in Java
Predefined methods
User-defined methods
(Programmer-defined methods)
Instance methods
Static methods
(Class methods)

Methods

User-defined method

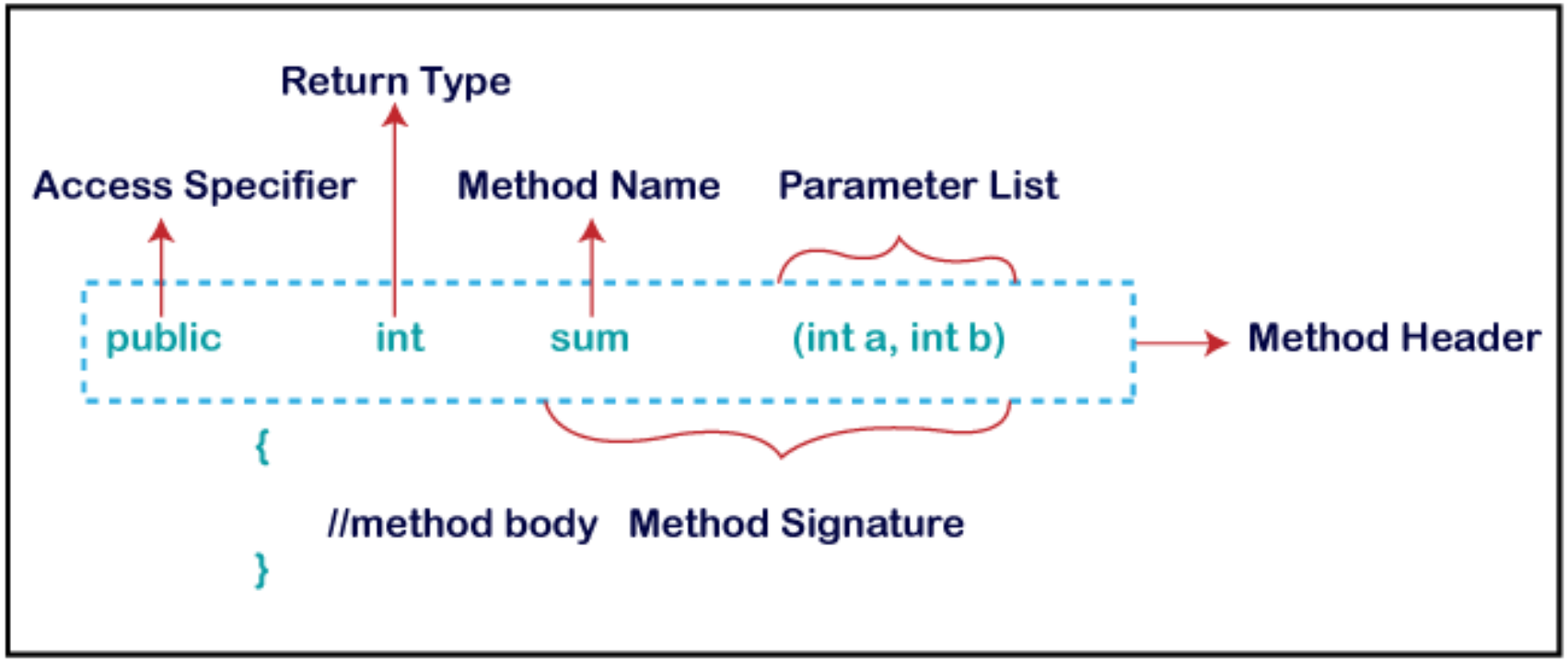
- **User-defined methods in Java** are those methods that are defined inside a class to perform a special task or function in an application.
- A user-defined method is also known as programmer-defined method. We can declare two types of user-defined methods in an application.

1. **Instance Method**
2. **Static Method**

Methods

Instance Method Declaration {Non-static Method}

- The method declaration provides information about method attributes, such as visibility, return-type, name, and arguments.
- It is accessed by using an object and .(dot) operator.



Methods

Calling Instance method

There are two steps to call an instance method in java.

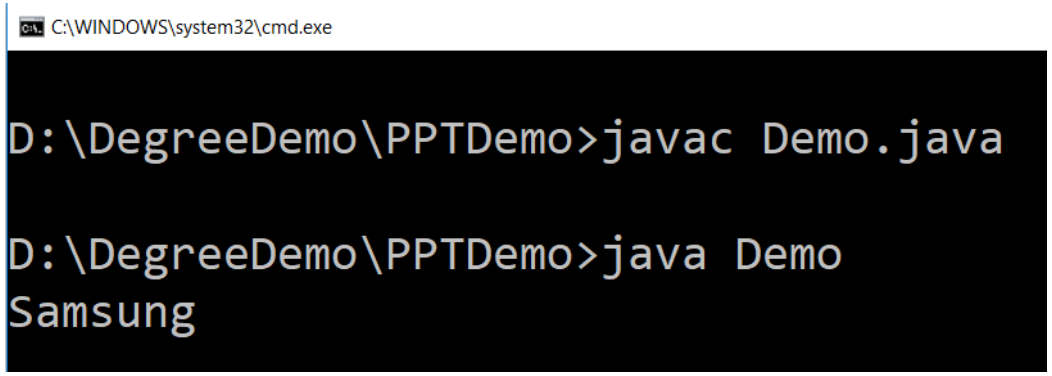
1. First, create an object of a class before calling an instance method of that class.
2. Second, call the method with a reference variable using the dot operator.

Example

```
//Creating an object of the class
InstanceMethodExample obj = new InstanceMethodExample();
//invoking instance method
System.out.println("The sum is: "+obj.add(12, 13));
```

Example (method)

```
public class SmartPhone {
    String manufacturer;
    String model;
    double storage;
    double screenSize;
    public String getManufacturer(){
        return manufacturer;
    }
    public void setManufacturer(String a){
        manufacturer = a;
    }
}
```




```
public class Demo {
    public static void main(String args[]) {
        SmartPhone sp = new SmartPhone();
        sp.setManufacturer("Samsung");  // calling method
        String name = sp.getManufacturer(); // calling method
        System.out.println(name);
    }
}
```

Methods

'this' keyword

- **'this'** is a reference variable that refers to the current object.

Usage:

- It can be used to refer current class instance variable.
- this() can be used to invoke current class constructor.
- It can be used to invoke current class method (implicitly)
- It can be passed as an argument in the method call.
- It can be passed as argument in the constructor call.
- It can also be used to return the current class instance.

Methods

'this' keyword

"**this**" keyword are used for two purpose, they are

- It always points to current class object.
- Whenever the formal parameter and data member of the class are similar and JVM gets an ambiguity (no clarity between formal parameter and data members of the class).

To differentiate between formal parameter and data member of the class, the data members of the class must be preceded by "**this**".

Syntax

```
this.data member of current class.
```

Methods

'this' keyword

Example of this keyword in java

```
class Employee
{
   int id;
   String name;

   Employee(int id,String name)
  {
   this.id = id;
   this.name = name;
  }
```

```
     void show()
  {
   System.out.println(id+" "+name);
  }
   public static void main(String args[])
   {
    Employee e1 = new Employee(111,"Harry");
    Employee e2 = new Employee(112,"Jacy");
    e1.show();
    e2.show();
   }
}
```

Output

```
111 Harry
112 Jacy
```

Practice Problems

- Program to find pass percentage of a student [Use 5 subject marks] using function
- Program to reverse a string using an user-defined function
- Program to get and display the mobile number of a person using function
- Program to check the input number positive or negative or zero using function
- Program to identify the highest frequency character in a string using function
- Program to print the last non-repeating character in a string using function

Constructor

- A *constructor* is a “special” member method which *initializes* the *objects* of class.

Properties of constructor:

- Constructor is *invoked automatically* whenever an object of class is created.
- *Constructor name* must be *same* as *class name*.
- Constructors do *not* have *return types* and they cannot return values, not even void.
- All classes have *constructors* by *default*: if you do not create a class constructor yourself, Java creates one for you. However, then you are not able to set initial values for object attributes.
- Types of Constructor
  1. Default Constructor
  2. Parameterized Constructor

Default Constructor

- Default constructor is the one which *invokes* by *default* when *object* of the class is *created*.
- It is generally used to *initialize* the *default value* of the data members.
- It is also called *no argument constructor.*

```
class Rect{
    int len;
    int wid;
    boolean isFilled;

    public Rect(){
        len = 6;
        wid = 6;
        isFilled = false;
    }
}
```

```
public class RectDemo{
    public static void main(String[] args) {
        Rect r1 = new Rect();
        System.out.println(r1.len);
    }
}
```

Parameterized Constructor

- *Constructors* that can take *arguments* are called parameterized constructors.
- Sometimes it is necessary to initialize the various data elements of different objects with different values when they are created.
- We can achieve this objective by passing arguments to the constructor when the objects are created.

```
class Rect{
    int len;
    int wid;
    boolean isFilled;

    public Rect(int l, int w, boolean isF){
        len = l;
        wid = w;
        isFilled = isF;
    }

}
```

```
public class RectDemo{
    public static void main(String[] args) {
        Rect r1 = new Rect(6,6,false);
        System.out.println(r1.len);
    }
}
```

Constructor

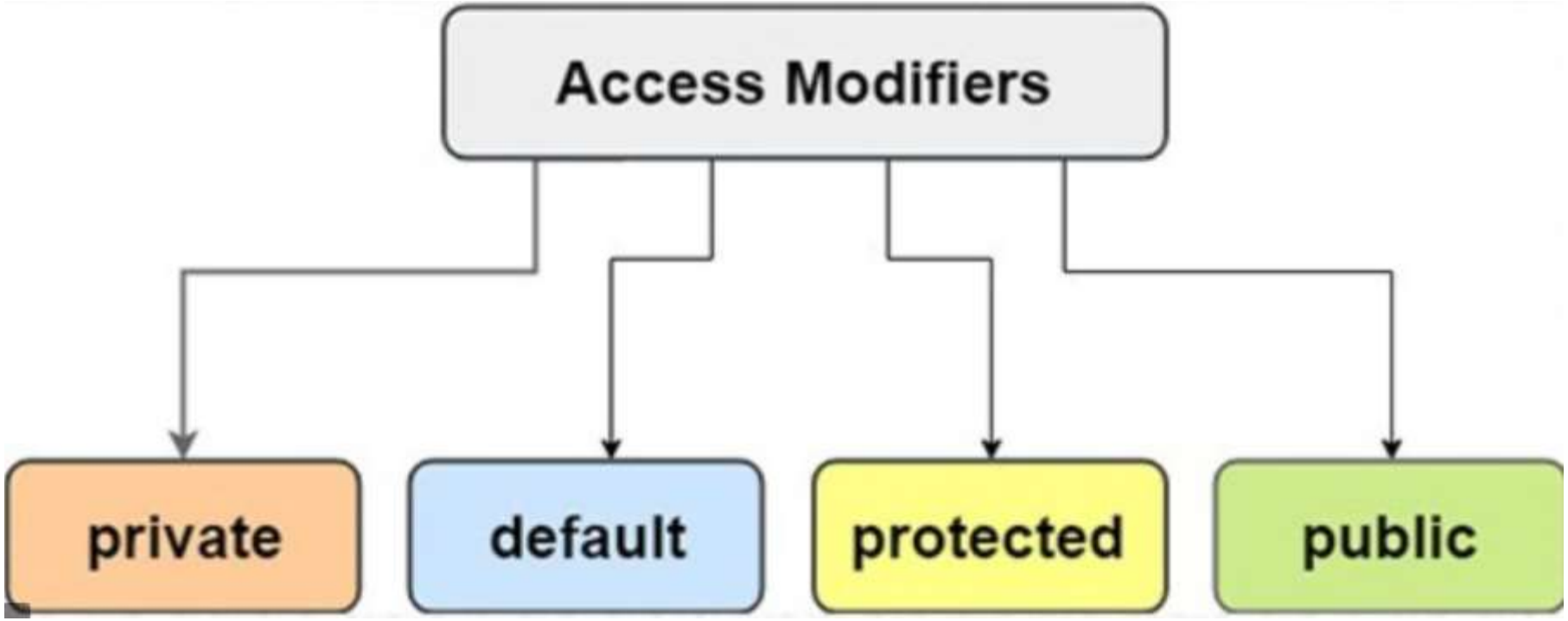
Constructor Overloading

- It is a technique of having more than one constructor with different parameter lists.

```
public class Employee {
 Employee(){
      System.out.println("Employee Details:");
 }
 Employee(String name){
      System.out.println("Employee name: " +name);
 }
 Employee(String nCompany, int id){
      System.out.println("Company name: " +nCompany);
      System.out.println("Employee id: " +id);
 }
}
public class Myclass {
public static void main(String[] args) {
  Employee emp=new Employee();
  Employee emp2=new Employee("Deep");
  Employee emp3=new Employee("HCL", 12234);
      }
}
```

Access Specifiers {Modifiers}

- Access Modifiers in Java helps to restrict the scope of a class, constructor, variable, method or data member.



Access Modifiers

Access Specifiers in Java

		public	private	protected	default
Same Package	Class	YES	YES	YES	YES
	Sub class	YES	NO	YES	YES
	Non sub class	YES	NO	YES	YES
Different Package	Sub class	YES	NO	YES	NO
	Non sub class	YES	NO	NO	NO

Access Modifiers – Public

- The keyword used for the public access specifier is ‘public’.
- Any variable, class, method, or any other member of a class that is declared as public will be accessible from anywhere in the program

```
package pkg1;
public class Calculate {
    public int additionOfNums(int firstNumber, int secondNumber) {
        return firstNumber+secondNumber;
    }
}
```

```
import pkg1.Calculate;
public class Program{
    public static void main(String agmts[]) {
        int A = 10;
        int B = 15;
        Calculate cal = new Calculate();
        int aplusb= cal.additionOfNums(A,B);

        //Printing A + B
        System.out.println("We get: " + aplusb);
    }
}
```

Access Modifiers – Private

- The keyword used for the private access specifier is '**private**'.
- Any variable, class, method, or any other member of a class that is declared as private will be accessible only within the class

```
package pkg1;
public class Calculate {
    private int sum(int a, int b) {
        return a+b;
    }
    public int additionOfNums(int firstNumber, int secondNumber) {
        return sum(firstNumber,secondNumber);
    }
}
```

```
import pkg1.Calculate;
public class Program{
    public static void main(String agmts[]) {
        int A = 76;
        int B = 14;
        // This will generate an error
        // int sum = cal.sum(A,B);

        // This is work
        Calculate cal = new Calculate();
        int sum = cal.additionOfNums(A,B);

        //Printing A + B
        System.out.println("We get: " + sum);
    }
}
```

Access Modifiers – default

- It doesn't require any keyword
- By default, all the members will be in default mode.
- The scope of the default access specifier is everywhere within the same package and not outside the package

```
package pkg1;

class Multiply {
    int multiply(int a, int b) {
        return a*b;
    }
}

class Calculate extends Multiply{
    // This class can access/inherit Multiply class
}
```

```
import pkg1.Calculate; // Cannot be imported
class Program extends Calculate{
    public static void main(String agmts[]) {
        // This program will generate an error because the Calculate class
        // Due to which it is only used/accessed in package pkg1
    }
}
```

Access Modifiers – protected

- The keyword used for the protected access specifier is '**protected**'.
- Any variable, class, method, or any other member of a class that is declared as protected will be accessible within the same package it is in as well as in other packages where it is being inherited.

```
package pkg1;
public class Calculate {
    protected int additionOfNums(int firstNumber, int secondNumber) {
        return firstNumber+secondNumber;
    }
}
```

```
import pkg1.Calculate;
public class Program extends Calculate{
    public static void main(String agmts[]) {
    int A = 64;
    int B = 13;

    // The add () method is not accessible directly
    Calculate cal = new Calculate();
    //int sum = cal.additionOfNums(A,B);

    // The Calculate class is inherited, so we can
    Program prog = new Program();
    int sum = prog.additionOfNums(A,B);

    //Printing A + B
    System.out.println("We get: " + sum);
    }
}
```

Static Members

- Static keyword can be used with class, variable, method and block.
- Static members belong to the class instead of a specific instance, this means if you make a member static, you can access it without object.

Static Variables

- Static variables are also called **class variables** because they can be accessed using class name.
- It occupy single location in the memory. i.e shared memory.
- These are accessible in both static and non-static methods, even non-static methods can change their values.
- Default value will be initialized by static variable

Static Members

Static Methods

- Static methods are also called class methods.
- A static method belongs to class, it can be used with class name directly. It is also accessible using instance references.
- Static methods can use static variables only, whereas non-static methods can use both instance variables and static variables.
- Generally, static methods are used to perform some common tasks for all objects of a class.

Static Members

```
class StaticDemo{
 static int a=10;  // static variable
static int b=20; // static variable
  public static void main(String [] args){
    System.out.println("a="+a);
   System.out.println("a="+b);
   show();
 }
 public static void show(){ // static method
    System.out.println("a="+a);
   System.out.println("a="+b);
}
}
```

Life time and scope:

- Static variable get life as soon as class is loaded into JVM and is available till class is removed from JVM or JVM is shutdown.
- And its scope is class scope means it is accessible throughout the class.

Static Members

Static Block

- Static block is used for initializing the static variables.
- This block gets executed when the class is loaded in the memory. i.e executed before main ( ).
- A class can have multiple static blocks, which will execute in the same sequence in which they have been written into the program.

Syntax

```
static
{
.....
//Set of Statements
.....
}
```

Static Members

Static Block

Example of Static Block

```
class StaticDemo
{
static
{
System.out.println("Hello how are u ?");
}
public static void main(String args[])
{
System.out.println("This is main()");
}
}
```

Output

```
Hello how are u ?
This is main()
```